Récapitulatif — Types, expressions, variables, traces - Solution

Partie A — Types et expressions

Exercice 1 : La fiche du livre

Une bibliothèque veut enregistrer ses livres. Quel type Python choisir (`bool` , `int` , `float` , `str`) pour stocker chaque information ?

- Le titre (ex: `"Le Petit Prince"`)

  `str`

- Le nombre de pages (ex: 96)

  `int`

- La note moyenne des lecteur·ices (ex: 4.5)

  `float`

- Le livre est-il emprunté ? (Oui/Non)

  `bool`

- Le code ISBN (ex: `"978-2-07-061275-8"`)

  `str`

L'ISBN contient des chiffres, mais aussi des tirets, et on ne fait jamais de calcul avec : c'est une chaîne de caractères.

Exercice 2 : La calculatrice de l'esprit

Prédire la valeur et le type Python de chaque expression :

Expression	Valeur	Type (int, float, str, bool)
`7 + 2`	`9`	`int`
`7 / 2`	`3.5`	`float`
`7 // 2`	`3`	`int`
`7 % 2`	`1`	`int`
`6 / 3`	`2.0`	`float`
`4 * 2.5`	`10.0`	`float`
`"7" + "2"`	`"72"`	`str`
`not (3 == 3)`	`False`	`bool`

Deux points à retenir : `/` donne **toujours** un `float` (même `6 / 3`), et dès qu'un `float` intervient dans un calcul le résultat est un `float`.

Exercice 3 : Vrai ou faux ?

Indiquer si chaque expression vaut `True` ou `False` :

- `"banane" < "cerise"`

  `True`

- `"chat" == "Chat"`

  `False`

- `10 == 10.0`

  `True`

- `"10" == 10`

  `False`

- `3 <= 3`

  `True`

- `not (2 > 5)`

  `True`

- Les chaînes se comparent dans l'ordre du dictionnaire, et les majuscules sont différentes des minuscules.
- `10 == 10.0` est vrai : c'est le même nombre. Mais `"10" == 10` est faux : un texte n'est pas un nombre.

Exercice 4 : Les équipes de la classe

Une classe compte 29 élèves. Le professeur veut former des équipes de 4.

- Quelle expression calcule le nombre d'équipes complètes ?

  `29 // 4`, ce qui vaut `7`

- Quelle expression calcule le nombre d'élèves qui restent sans équipe ?

  `29 % 4`, ce qui vaut `1`

- Écrire une expression booléenne qui vaut `True` si tous les élèves ont une équipe

  `29 % 4 == 0` (ici `False`)

Exercice 5 : Analyse d'erreurs

Chacune de ces expressions provoque une erreur. La corriger en utilisant une conversion `int()`, `float()` ou `str()`, puis donner la valeur obtenue.

Expression	Expression corrigée	Valeur
`"3" + 4`	`int("3") + 4`	`7`

`"Âge : " + 15`	`"Âge : " + str(15)`	`"Âge : 15"`
`int("3.5") + 1`	`float("3.5") + 1`	`4.5`
`2,5 + 1`	`2.5 + 1`	`3.5`

`int("3.5")` échoue : `int()` n'accepte que du texte contenant un nombre entier. Et en Python le séparateur décimal est le point, jamais la virgule.

Exercice 6 : Le coffre-fort

Un coffre s'ouvre si l'expression suivante vaut `True` :

`not (18 % 3 == 1) and (3**2 >= 9)`

Donner la valeur de chaque étape :

- `18 % 3`

  `0`
- `18 % 3 == 1`

  `False`
- `not (18 % 3 == 1)`

  `True`
- `3**2`

  `9`
- `3**2 >= 9`

  `True`
- L'expression complète : le coffre s'ouvre-t-il ?

  `True`, le coffre s'ouvre

Exercice 7 : L'ordre des priorités

Ces deux expressions ne donnent pas le même résultat. **Décomposer** chaque *évaluation* :

- `not (True and False)`

  `not False → True`
- `not True and False`

  `(not True) and False → False and False → False`

Pourquoi ?

`not` est prioritaire sur `and` : sans parenthèses, `not` ne s'applique qu'à `True`.

Partie B — Variables et traces

Exercice 8 : Le robot aspirateur

Écrire la trace de l'exécution de ce programme :

```
# État initial du robot
batterie = 20
en_charge = False

batterie = batterie - 12   # nettoie le salon
en_charge = not en_charge  # retourne à sa base
batterie = batterie + 15   # se recharge
en_charge = not en_charge  # repart travailler
batterie = batterie - 3    # nettoie la cuisine
travail_fini = (batterie > 15) and (en_charge == False)
print(travail_fini)
```

ligne	batterie	en_charge	travail_fini	affichage
2	`20`			
3		`False`		
5	`8`			
6		`True`		
7	`23`			
8		`False`		
9	`20`			
10			`True`	
11				`True`

Exercice 9 : Que s'affiche-t-il ?

Écrire ce que chaque `print` affiche.

Bien regarder le type de chaque élément : il est possible qu'une ligne provoque une erreur.

```
score = 3
print("score")                    _____________________________
score = score * 10
print(score)                      _____________________________
score = score + 5
print(score)                      _____________________________
print("score = " + str(score))    _____________________________
print("score = " + score)         _____________________________
```

ligne	affiche

2	`score` (le texte, pas la valeur de la variable)
4	`30`
6	`35`
7	`score = 35`
8	`TypeError: on ne peut pas concaténer str et int`

Exercice 10 : Le bulletin météo

Écrire le code python qui :

1. Crée ces 3 variables et leur assigne leurs valeurs respectives :
   - `ville` (str) : `"Genève"`
   - `température` (float) : `21.5`
   - `pluie` (bool) : `False`
2. Crée une variable `bulletin` *(en utilisant les variables, pas des valeurs fixes)* qui contient : `Genève | 21.5 degrés | pluie: False`
3. Affiche la valeur de cette variable

Bien regarder le type de chaque élément : il va falloir des conversions avant de concaténer.

```python
ville = "Genève"
température = 21.5
pluie = False
bulletin = ville + " | " + str(température) + " degrés | pluie: " + str(pluie)
print(bulletin)
```

Exercice 11 : Le tarif réduit

Un cinéma accorde le tarif réduit aux étudiants et aux retraités. Le statut du client est stocké dans une variable `statut`, qui peut valoir `"étudiant"`, `"retraité"`, `"actif"`

Écrire le programme qui :

- Stocke le statut du client dans la variable `statut` (choisir une valeur parmi les trois)
- Détermine dans une variable `tarif_réduit` s'il faut appliquer le tarif réduit
- Affiche cette valeur (`True` s'il faut appliquer le tarif réduit, `False` sinon)

```python
statut = "retraité"
tarif_réduit = statut == "étudiant" or statut == "retraité"
print(tarif_réduit)
```

Exercice 12 : Échanger la valeur de deux variables

On aimerait échanger la *valeur* de deux variables :

```python
a = "sel"
b = "poivre"

# Écrire le code pour échanger a et b, sans réécrire "sel" ni "poivre" :
```

```
...

print(a) # doit afficher poivre
print(b) # doit afficher sel
```

Faire une trace de votre solution pour la vérifier : il est très probable que votre premier jet ne fasse pas ce que vous voulez.

```
# Il faut une troisième variable :
# si on commence par écrire a = b, la valeur "sel" est perdue pour toujours.
c = b
b = a
a = c
```